

PhanMeeEin AI Component Presentation Guidebook

Beginner-friendly prep notes for RAG, tool calling, model selection, key usage, main-app boundaries, DB memory, and storage

One-sentence pitch

The browser talks only to the main Laravel app; the main app protects users and data; the AI package talks to Gemini; Gemini may request Brave Search as a tool; PHP executes the tool and Gemini writes the final answer.

What the examiner should remember

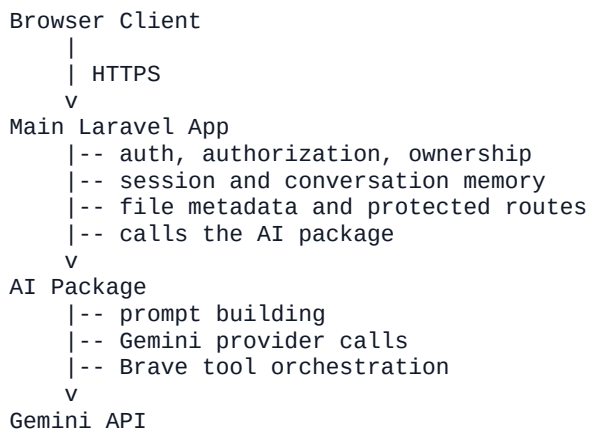
1. Main app = security boundary and owner of user data.
2. AI package = prompt building, model calls, and tool orchestration.
3. Gemini decides whether a tool is needed; PHP executes the tool.
4. MySQL stores authoritative chat memory; the session stores only the active conversation ID.
5. R2 stores files; Brave Search provides fresh facts; RAG helps with local website knowledge.

How to use this guide

Read the sections in order once, then use the slide order and Q&A; pages as your rehearsal checklist. If you can explain each section in one short sentence, you are ready for the presentation.

1. Architecture story you should tell first

Start with the big picture, not the model names. The audience should first understand the trust boundary and the data flow.



Simple explanation you can say out loud:

"The browser never touches Gemini or Brave directly. The main Laravel app sits in the middle and protects the user. The AI package handles the model work. Gemini can ask for a live web search tool, and the PHP code runs that tool safely."

Relevant code files to mention:

- routes/user.php - the guest page and user routes
- resources/views/user/layout/master.blade.php - where the widget is included
- packages/Local/AiCompanion/src/Http/Controllers/AiChatController.php - request entry point
- packages/Local/AiCompanion/src/Services/GeminiClient.php - model calls and tool loop
- packages/Local/AiCompanion/src/Services/BraveSearchClient.php - live web search client
- packages/Local/AiCompanion/src/Services/PromptBuilder.php - prompt assembly
- packages/Local/AiCompanion/config/ai-companion.php - env-driven settings

2. What each layer is responsible for

Use this section if someone asks "Why did you split it this way?"

Main Laravel app

- Authenticates the user.
- Checks authorization and ownership.
- Stores authoritative conversation history in MySQL.
- Serves the floating chat widget into the page.
- Keeps secrets server-side and controls access to the AI service.

AI package

- Builds the prompt.
- Sends messages to Gemini.
- Defines allowed tools.
- Executes Brave Search only when Gemini requests it.
- Handles model selection, key rotation, and fallback logic.

Gemini API

- Generates the answer.
- May request the brave_search tool.
- Does not directly run PHP.
- Does not see your server secrets.

Aiven DB and Cloudflare R2

- Aiven DB stores users, sessions, chat rows, and other app data.
- R2 stores uploaded content and resources.
- The main app decides when data is read or written.

3. RAG in this project

RAG means Retrieval Augmented Generation. In plain words, it means the model gets extra useful text before it answers.

In your project there are two retrieval paths:

- Local help context from webhelper.md for website usage questions.
- Live Brave Search results for fresh facts and time-sensitive questions.

Why it exists

- It keeps the AI grounded in your project knowledge.
- It helps the AI answer website-help questions more accurately.
- It avoids guessing when the answer depends on current information.

What to say in the presentation

"Our RAG approach is small and practical. We do not dump everything into the model. We give it the webapp help text when needed, and for current facts we let Gemini request a Brave Search tool call."

Important difference

- RAG is about giving the model retrieved context.
- Tool calling is about letting the model request an external function.
- They work together, but they are not the same thing.

Current code idea

```
PromptBuilder
-> adds webapp context from webhelper.md
-> adds the user message and conversation history
GeminiClient
-> sends prompt + brave_search tool schema
-> if Gemini requests the tool, PHP runs BraveSearchClient
-> send functionResponse back to Gemini
```

4. Tool calling, explained like a beginner

This is the most important part of your talk. Say it slowly and clearly.

```
User message
-> Gemini sees the prompt and the allowed tool list
-> Gemini decides whether brave_search is useful
-> Gemini returns functionCall with a query
-> PHP executes BraveSearchClient
-> PHP sends functionResponse back
-> Gemini writes the final answer
```

The key idea

The model decides. The code executes.

In your current code this happens inside `GeminiClient::generateWithBraveSearchTool()`.

How to explain it verbally

- The model is like a smart planner.
- Your PHP code is the worker that actually does the API call.
- The model cannot directly call Brave or PHP by itself.
- The model only asks for the tool in a structured response.

If asked how this is different from the old keyword version:

- Old version: PHP looked for keywords and decided first.
- New version: Gemini decides if a tool is needed.
- That makes the flow more flexible and more like real agentic AI.

Good sentence for the examiner

"We do not hard-code the freshness decision with keywords anymore. We allow Gemini to request a search tool, and the Laravel package safely executes only that allow-listed tool."

5. Model selection and key usage

Keep this explanation simple: model selection is resilience, not decoration.

Current settings you should know

Variable	Meaning	Why it matters
GEMINI_API_KEY_1/2/3	Multiple Gemini keys	Lets the app rotate or fail over if one key is limited or unavailable.
AI_COMPANION_MODEL	Primary model	First choice for normal chat responses.
AI_COMPANION_FALLBACK_MODEL	Backup model	Used if the primary model fails.
AI_COMPANION_TIMEOUT	Request timeout	Prevents hanging requests.
BRAVE_SEARCH_API_KEY	Brave API key	Allows the tool call to fetch live web context.

How to explain key usage safely

- All keys stay on the server.
- The browser never sees them.
- The app can keep working if one model or key has trouble.
- This is not about giving the model direct access to secrets.

Good phrase to remember

"The key rotation is for reliability. The keys never leave the backend."

6. Database, session, and storage boundaries

This is where you sound responsible and safe. The examiner will likely care about this.

Database memory

- The main app stores authoritative AI chat history in MySQL.
- Tables used: ai_conversations and ai_messages.
- This means refreshes and reconnects do not lose the real conversation history.
- The session stores only the active conversation ID and lightweight UI state.

Storage boundaries

- R2 stores uploaded images and resources.
- Laravel checks access before serving private files.
- The AI package does not own the file store.
- The browser should not directly access protected services.

Security line you should say

"The main Laravel app is the trust boundary. It controls who can talk to the AI service, who can see their chat history, and who can access uploaded files."

Why not store memory in the browser?

- It is not authoritative.
- It can be cleared by the user.
- It is harder to control and audit.
- The project specification says the main app should own the real memory.

Why not let Gemini own the DB?

- The AI service should remain stateless where possible.
- User authentication and data ownership belong to the main app.
- That keeps the architecture simpler and safer.

7. Suggested presentation script

Use this as a rehearsal script. Do not memorize every word. Understand the order.

Opening - 20 seconds

"This project uses a secure main Laravel app and a separate AI package. The main app owns users, chat history, and file access. The AI package handles Gemini, RAG, and live web search when the model asks for it."

Middle - 1 to 2 minutes

- Show the architecture diagram.
- Point out the trust boundary around the main app.
- Explain that RAG gives the model helpful context.
- Explain that Gemini can request the brave_search tool.
- Say that PHP executes the tool and sends the result back.

Model and keys - 30 seconds

"We use primary and fallback Gemini models, plus multiple keys for resilience. The keys stay server-side, and the browser never sees them."

Database and storage - 30 seconds

"The chat memory is stored in MySQL, the session only keeps the active conversation pointer, and uploaded files stay in R2 behind Laravel access checks."

Closing - 15 seconds

"So the main app stays in control, the AI package stays modular, and the system can answer both learning questions and fresh web questions without exposing secrets."

If you forget a detail during the talk, return to the simpler message: main app protects, AI package orchestrates, Gemini decides, PHP executes.

8. Common questions and short answers

Memorize these short answers. They are the kind of questions people ask when they want to check if you understand the architecture.

Q: Why not call Brave directly from the browser?

A: Because the Brave API key must stay secret and the browser should not reach external services directly. The main app is the security boundary.

Q: Does Gemini directly run the PHP code?

A: No. Gemini only returns a function call. PHP reads that call and runs the Brave client.

Q: Why do you need RAG and tool calling both?

A: RAG gives local project context. Tool calling gives fresh live facts. They solve different problems.

Q: Why multiple Gemini keys?

A: For resilience and failover if a key hits limits or has a temporary issue.

Q: Why store chat history in MySQL?

A: Because it is the authoritative record and survives refreshes, restarts, and browser changes.

Q: What is the simplest one-line summary of the AI architecture?

A: The main app protects the user, the AI package orchestrates the model, and Gemini can request Brave Search through a safe server-side tool call.

Final rehearsal checklist

- Can you explain the trust boundary in one sentence?
- Can you explain RAG without using jargon?
- Can you explain why model selection and key rotation exist?
- Can you explain the difference between the model deciding and PHP executing?
- Can you explain where chat memory lives and why?

Code map for last-minute review

```
packages/Local/AiCompanion/src/Http/Controllers/AiChatController.php
packages/Local/AiCompanion/src/Services/GeminiClient.php
packages/Local/AiCompanion/src/Services/BraveSearchClient.php
packages/Local/AiCompanion/src/Services/PromptBuilder.php
packages/Local/AiCompanion/config/ai-companion.php
packages/Local/AiCompanion/resources/views/widget.blade.php
resources/views/user/layout/master.blade.php
```

End with confidence: the system is not "AI everywhere". It is a controlled Laravel architecture with a clear boundary, a small AI package, and server-side tool use.